

Chapter 1

Designing Functions

In mathematics, a *function* is a mapping from input values in some *domain* to output values in some *codomain*. The domain and codomain of a function are the types of values that can appear, while the actual values are members of those types. The *range* of a function is the collection of values in the codomain that actually appear as the result from some input value. Every member of the domain is necessarily mapped to some member of the range.

In a programming language like Java, usually there are only a limited quantity of types available, and you must choose the best approximation of your desired domain and range. In §0.5 we wrote a squaring function that takes an `int` as input and yields an `int` as output. However, that choice of codomain is not the best, as it does not contain the entire range of values that could be produced. The largest `int` is approximately 2 billion, but squaring 50_000 should yield 2.5 billion. Arithmetic operations on these fixed-size primitive types will silently wrap around back into the negatives! But even the square of the largest possible `int` does not exceed the potential range of a `long` value, so a better codomain (return type) would be `long`.

<int-to-long square 13>≡

```
public static long square(int x) {  
    return ((long)x)*((long)x);  
}
```

13

If you find that your codomain cannot contain all possible results, there are a couple options available. First, if possible, you can extend your codomain by selecting a larger type. That is what we did here. Alternatively, you can restrict the domain by documenting that the real domain is something smaller than that implied by the input type restrictions and throwing an exception when given an impermissible input. Doing neither is a surefire way to have buggy, incorrect software. We shall discuss exceptions further in §1.2

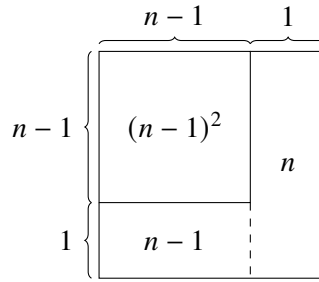


Figure 1.1: Squaring in unary.

1.1 Recursive Functions

A core tenet of function design is that the shape of your function should follow the shape of your data. If your data can be broken down into smaller pieces of data of the same kind, then a *recursive function* is probably a good approach. *Recursion* is when something is defined in terms of itself; a recursive function is a function that operates by breaking down the input data into smaller pieces of data of the same kind, solving those smaller problems, then combining the solutions to get a solution to the full problem. This relies on some problems being small enough to be solved trivially, having their outputs listed directly. These are the *base cases* of the recursion.

We'll rewrite the integer squaring function in recursive form to demonstrate. For now, we assume that input will be nonnegative. The negative case will be handled later. First, if we are to define a recursive function here, we should describe some way in which a number can be broken down into one or more smaller numbers. One representation of nonnegative integers is as a sequence of tally marks. For instance, 4 can be represented by |||| and by removing one mark, we get ||| which represents 3. The question then becomes, how do we get from 3^2 to 4^2 , or in general, how do we get from $(n-1)^2$ to n^2 ? Figure 1.1 shows the general picture: we first extend one edge of an $(n-1) \times (n-1)$ square (adding $n-1$) to have an $n \times (n-1)$ rectangle, then extend the long edge of the result (adding n) to get an $n \times n$ square. The base case is a square of side length 0, where $0^2 = 0$. The conversions to `long` in the following code could be omitted, as Java will automatically *widen* a smaller type to a larger type used in the same expression, but leaving them explicit makes clear that a conversion is happening.

```

14 <int-to-long squaring 14>≡
    public static long squareUnary(int n) {
        if (n == 0) { return 0L; }
        return squareUnary(n - 1) + (long)n + (long)n-1L;
    }
15>
```

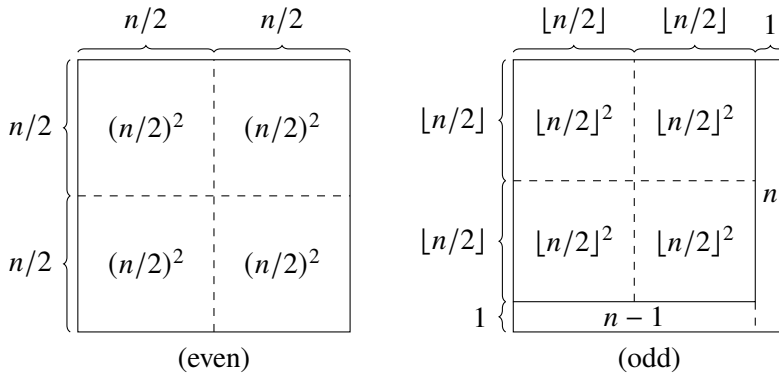


Figure 1.2: Squaring in binary.

Alternatively, we might represent our input in some place-value system. Binary numbers are commonly used in the context of computer science, as components are in one of two states, either powered or unpowered. A binary digit, also known as a *bit*, is either zero or one. The value of a one-bit number is just the value of that bit. The value of a larger number is the value of its rightmost (least-significant) bit plus twice the value of the number formed by removing that bit. For example, 1_2 is just 1. So 10_2 is $0 + 2 \cdot 1 = 2$, and then 101_2 is $1 + 2 \cdot 2 = 5$, and then 1011_2 is $1 + 2 \cdot 5 = 11$. The rightmost bit of n tells whether n is even or odd, and the number obtained by deleting that bit is $\lfloor n/2 \rfloor$, the quotient n divided by 2, rounded down. Treating the number this way, we have to care what the value of the rightmost bit is, but we can make fewer recursive calls, as each call cuts n in half instead of merely decrementing it by one. Figure 1.2 demonstrates the possibilities. Note that in Java, the expression $n/2$ computes $\lfloor n/2 \rfloor$ if n is a positive value of integer type like `int` or `long`.

<int-to-long squaring 14>+≡

◀14 15

```
public static long squareBinary(int n) {
    if (n == 0) { return 0L; }
    if (n == 1) { return 1L; }
    if (n%2 == 0) { return 4L*squareBinary(n/2); }
    return 4L*squareBinary(n/2) + (long)n + (long)n-1L;
}
```

If we call `squareUnary(42)`, this internally calls `squareUnary(41)`, which in turn calls `squareUnary(40)`, and so on, until finally the base case `squareUnary(0)` is called, for a total of 43 calls to `squareUnary`. In contrast, `squareBinary(42)` makes internal calls with n being just 21, 10, 5, 2, and 1, for a total of 6 calls. The deepest possible chain of calls for any positive `int` for `squareBinary` is just 31, while `squareUnary` may make billions.

1.2 Exceptions

Sometimes, the domain of a function does not naturally fill any Java type. In those cases, we must restrict the domain in another way: by documenting the restriction and throwing an appropriate exception. An *exception* is an event that disrupts the normal flow of execution. There are three kinds of exceptions:

- *Checked exceptions* indicate that the environment is wrong. The operation could, in principle, succeed, except that some external state is inappropriate. For example, attempting to open a file that does not exist for reading may result in a checked `FileNotFoundException`, because a file by that name could exist, but does not.
- *Errors* indicate that the system is in a state from which it cannot reasonably recover. For instance, after successfully opening a file for reading, if the system detects read failures from the drive it is on, attempting to read from the file may result in an `IOException`. You cannot be expected to recover from external system failure.
- *Unchecked exceptions* indicate that the program is wrong. For example, calling a method on a null reference yields a `NullPointerException`. Attempting to do so could never be appropriate.

What we want here is an unchecked exception: calling a function with a value that is not in its domain is a logic error of your program and is not something you should attempt to recover from. Some of the more common types of unchecked exceptions are as follows, all deriving from `RuntimeException`:

- `ArithmeticException`: illegal arithmetic, such as division by zero, was attempted.
- `IllegalArgumentException`: an inappropriate argument was provided.
- `IllegalStateException`: a method has been called at an inappropriate time.
- `IndexOutOfBoundsException`: an index (to an array, list, or suchlike) is not in the appropriate range.
- `NoSuchElementException`: the element being requested does not exist.
- `NullPointerException` attempt to use `null` where an object is required.

If we want to say that negative side lengths do not exist, and thus our squaring methods should not be called with negative arguments, the best choice of exception

would be an `IllegalArgumentException`. Each of these can be created in one of two ways: you can use something like `new IllegalArgumentException()` to create one with no extra details, or you can pass a `String` argument to supply extra details. For instance, we might modify `squareBinary` to forbid negative parameters as follows. I include a Javadoc documentation comment to demonstrate how these domain restrictions should be documented, using the “`@throws`” tag.

<int-to-long squaring with exceptions 17>≡

17

```
/**
 * Returns the area of a square with the given side length.
 * @param n the side length
 * @return the square of {@code n}
 * @throws IllegalArgumentException if {@code n} is negative
 */
public static long squareBinary(int n) {
    if (n < 0) {
        throw new IllegalArgumentException(
            "negative lengths do not exist"
        );
    }
    if (n == 0) { return 0L; }
    if (n == 1) { return 1L; }
    if (n%2 == 0) { return 4L*squareBinary(n/2); }
    return 4L*squareBinary(n/2) + (long)n + (long)n-1L;
}
```

Knowing that $(-n)^2 = n^2$, we might instead attempt to account for negative arguments by simply returning `squareBinary(-n)` for that case rather than throwing an exception. There is, however, a subtle issue with that approach. Look carefully at the range of an `int` as given in §0.3, and see if you can figure out what the issue is. Further, see if you can find a good fix that does not involve changing the type or hardcoding any large values.

1.3 Iteration and Tail-Recursive Form

A *tail-call* is a call to a function after which the calling function has no more work to do. An optimization supported by many compiled programming languages (but not Java) called *tail-call optimization* removes the call, essentially copy-pasting the body of the called function in place of the call-site. If all recursive calls in a function are tail-calls, then the function is *tail-recursive*. Tail-call optimization for tail-recursive functions is translation to a looping construct. We first discuss tail-recursive form and how to attain it, then how to do this optimization manually.

Our squaring functions to this point have not been tail-recursive, as after the result of the subproblem is found, there is still arithmetic to be done on the

result. Consider `squareUnary`: for the recursive case, it first finds the value of `squareUnary(n - 1)`, then adds `(long)n + (long)n-1L` to that result. We want to somehow move this addition to the inside of the parentheses of the function call, so that nothing happens after the call. To do so, we move the implementation to a private helper function that takes a second argument: a partial result. As we are adding into this partial result, its initial value should be 0, the additive identity. Then, when reaching the base case, we return the partial result plus whatever value would normally be returned (in this case, zero).

```
18a <int-to-long squaring with tail-recursion 18a>≡ 18b>
    public static long squareUnary(int n) {
        return squareUnary(n, 0L);
    }
    private static long squareUnary(int n, long partial) {
        if (n == 0) { return partial; }
        return squareUnary(n - 1, partial + (long)n + (long)n-1L);
    }
```

The conversion for `squareBinary` is more complicated. In the even case, it multiplies the solution to its subproblem by four. In the odd case, it does this and adds some extra content. We can achieve the desired result by using two extra variables: the partial result that information is added into and the current multiplier. The default multiplier is not 0 but 1, the multiplicative identity.

```
18b <int-to-long squaring with tail-recursion 18a>+≡ <18a
    public static long squareBinary(int n) {
        return squareBinary(n, 1L, 0L);
    }
    private static long squareBinary(int n, long mul, long partial) {
        if (n == 0) { return partial; }
        if (n == 1) { return partial + mul; }
        if (n%2 == 0) { return squareBinary(n/2, 4L*mul, partial); }
        return squareBinary(n/2,
            4L*mul,
            partial + mul*((long)n + (long)n-1L));
    }
```

As mentioned, tail-call optimization works by transferring execution directly to the target function, rather than going through the call/return system. (This is actually why Java does not support tail-call optimization: it removes information about the calling function, which might be wanted if an exception is thrown.) Before execution can transfer to the target function, the parameters need to be set appropriately. As the tail-call is a recursive call, the target is the top of `squareBinary` itself; this is exactly the job of a while-loop. The trivial form of the optimization wraps the entire function body in a `while (true) { ... }` loop, and replaces recursive calls with simple updates to the variables involved. Notice that the `else` is necessary for the odd case now, as the even case no longer uses `return`.

<squareBinary helper in loop form 19a>≡

19a

```
public static long squareBinary(int n, long mul, long partial) {
    while (true) {
        if (n == 0) { return partial; }
        if (n == 1) { return partial + mul; }
        if (n%2 == 0) {
            mul = 4L*mul;
            n = n/2;
        } else {
            partial = partial + mul*((long)n + (long)n-1L);
            mul = 4L*mul;
            n = n/2;
        }
    }
}
```

Notice that the computation of `partial` involves both `mul` and `n`. For this reason, the update order is important: `partial` must be updated before either of `mul` or `n`.

In this form, only one call is ever made into this function; this preserves a great deal of space in the call-stack. This direct translation, however, leaves much to be desired. For instance, both the even and odd recursive case make the same changes to `mul` and `n`. And the `while (true) { ... }` structure makes it look like this loop will go on forever; instead we can move the base cases to below the loop and have the condition be “is not a base case”. Here the base cases are at 0 and 1, so the condition could be something like `while (n != 0 && n != 1)`, but under the assumption that inputs are nonnegative it is simpler to say `while (n > 1)`.

<squareBinary helper in better loop form 19b>≡

19b

```
public static long squareBinary(int n, long mul, long partial) {
    while (n > 1) {
        if (n%2 == 1) {
            partial = partial + mul*((long)n + (long)n-1L);
        }
        mul = 4L*mul;
        n = n/2;
    }
    if (n == 0) { return partial; }
    return partial + mul;
}
```

This is a reasonable form for the private helper function. It does not look like it will loop forever, and there is no unnecessary duplication of code inside the loop. However, this helper function is not needed at all anymore. The call to the helper is a tail-call, so it can be replaced by instantiating the relevant variables (`mul` and `partial`) and pasting the body of the function in place of the call. As

nothing references the helper anymore, it can be discarded. After making all of these changes, the final form of the public function is as follows.

20a *<squareBinary in loop form 20a>*≡

```
public static long squareBinary(int n) {
    long mul = 1L;
    long partial = 0L;
    while (n > 1) {
        if (n%2 == 1) {
            partial = partial + mul*((long)n + (long)n-1L);
        }
        mul = 4L*mul;
        n = n/2;
    }
    if (n == 0) { return partial; }
    return partial + mul;
}
```

Doing the same for the unary version gives the following.

20b *<squareUnary in loop form 20b>*≡

```
public static long squareUnary(int n) {
    long partial = 0L;
    while (n != 0) {
        partial = partial + (long)n + (long)n-1L;
        n = n - 1;
    }
    return partial;
}
```

In either case, we can throw an exception for negative parameters as discussed in the previous section.

1.4 Summary

Translating a function definition from the pure world of mathematics into efficient program code requires careful thought. You should ensure that your domain (parameter type restrictions) and codomain (return type) are sensible: the codomain should actually contain the output corresponding to each valid input, and all possible inputs should be valid. In cases where it is truly impossible for all inputs to be valid, you should document the true domain and raise an appropriate exception upon receiving invalid input.

Recursion is a powerful tool of thought. If you initially write your functions such that the shape of the function matches the shape of the data, then you will have a much easier time than if you were to force a function structure onto mismatched

data. Translating a well-reasoned recursive solution into a loop structure is an optimization that can be done after you are sure that the solution works.

Numeric inputs are interesting, because numbers exist in the abstract devoid of representation. Any representation that we choose is something we have imposed. The unary representation based on tally marks points to solutions built upon subtracting one from the input. The binary representation points to solutions built upon halving the input. You have the freedom to impose whatever representation is most convenient to the problem at hand. In future chapters, we will discuss concrete data structures, which exist with a single, specified representation. Oftentimes, there will be more than one concrete data structure that is capable of representing your desired values, and you will need to analyze how the representation influences your solution.